esiea

ANGULAR

Modèle et Services

Ressources :

https://angular.io/docs

Modèle de données

esiea

- Afin de pouvoir partager les données dans notre application, nous allons créer un modèle objet.
- Il s'agit d'encapsuler les différentes données affichées dans des classes partagées dans toute l'application.
- Nous allons commencer par créer une carte :

```
ng g class model/Card
```

- Voici le résultat :

```
CREATE src/app/model/card.spec.ts (146 bytes)
CREATE src/app/model/card.ts (22 bytes)
```

- Le fichier **`card.ts`** :

```
export class Card {
}
```

Modèle de données

esiea

```
export class Card {
    id : number;
    firsName : string;
    lastName : string;
    birthYear : number;
    imgUrl : string;
}
```

- Les attributs sont en erreur car non initialisés. Il y a trois solutions pour corriger le problème :

1)Initialiser chaque attribut lors de sa définition :

```
firstName : string = "";
```

2)Initialiser les attributs dans le constructeur :

```
constructor(id : number, firsName : string, lastName : string,birthYear : number,imgUrl : string) {
    this.id = id;
    /// ... Suite
}
```

3)Spécifier que ces attributs peuvent être non définis en les suffixant du caractère « ! » :

```
id! : number;
```

Affichage

esiea

- Pour utiliser les données dans un composant, nous allons définir un composant et lui associer une instance de fiche.

- Créons un composant chargé d'afficher une fiche. Appelons-le **`card-detail`**

```
ng g c components/card/card-detail
```

- Notez que ce composant a été généré dans un sous dossier « **`card`** » qui contiendra aussi, à terme, les composants chargés d'afficher les cartes dans une liste.

Affichage

esiea

- Pour tester, on initialise notre fiche dans le constructeur du composant.

```
export class CardDetail {
  card : Card;
  constructor() {
    this.card = new Card(1, "Dupont", "Antoine", 1996,
        "https://static.actu.fr/uploads/2023/10/antoine-dupont-1.png");
   }
}
```

On constate ici que le choix a été fait d'initialiser les données de la classe `Card` via son constructeur.

- Partie affichage de notre composant :

```
<div style="border: solid 2px #36a9e1;">
  <p>Nom : {{card.lastName}}</p>
  <p>Prénom : {{card.firsName}}</p>
  <p>Année de naissance : {{card.birthYear}}</p>
  <img [src]="card.imgUrl" style="max-width: 200px;">
</div>
```

Nom : Antoine

Prénom : Dupont

Année de naissance : 1996

Injection de données

esiea

- Pour que notre composant soit utile, il est nécessaire de le « paramétrer » avec une fiche particulière afin de pouvoir utiliser le même composant pour afficher plusieurs fiches.

- Le mécanisme de l'injection de données est utilisé.

- En Angular, ça se traduit par la directive `@Input` dans le script du composant chargé d'afficher des éléments du modèle.

Injection de données

esiea

- Cela se traduit par les changements suivants dans le **`ts`** :

```
export class CardDetailComponent {
  @Input()
  card! : Card;

  constructor() {
  }
}
```

Annotation **`@Input()`**

`card` est défini comme facultatif (**`!`**) pour permettre de l'injecter par la suite.

- On va ensuite, pour tester l'injection, créer un ensemble de fiches dans le constructeur de notre **`app.ts`**. :

```
cards : Card[];
constructor() {
  this.cards = [
    new Card(1, "Dupont", "Antoine", 1996, "https://static.actu.fr/uploads/2023/10/antoine-dupont-1.png"),
    new Card(2, "Ntamack", "Romain", 1999, "https://static.actu.fr/upload/250x333/romain-ntamack-250.jpg"),
    new Card(3, "Ramos", "Thomas", 1995,  "https://static.actu.fr/uploads/2022/12/OM134361.jpg"),
    new Card(4, "Bielle-Biarrey", "Louis", 2003, "https://static.actu.fr/uploads/2024/bielle-biarrey.jpg"),
  ]
}
```

Injection de données

esiea

- On peut alors utiliser le `@for` et le property binding dans l'HTML :

```
<div style="display:inline-flex">
@for (card of cards; track card.id) {
    <app-card-detail [card]="card"></app-card-detail>
}
</div>
```

On note que l'annotation `@Input` dans le `ts` transforme l'attribut annoté en une propriété dans l'HTML. On utilise donc le property binding.

Nom : Antoine	Nom : Romain	Nom : Thomas	Nom : Louis
Prénom : Dupont	Prénom : Ntamack	Prénom : Ramos	Prénom : Bielle-Biarrey
Année de naissance : 1996	Année de naissance : 1999	Année de naissance : 1995	Année de naissance : 2003

Services

esiea

- Les services permettent de centraliser l'accès aux données dans l'application.
- Les avantages sont nombreux :
  - ne pas avoir de duplication de code à différents niveaux de l'application.
  - facilite la maintenance, la lisibilité, la stabilité du code.
  - évite les erreurs en centralisant les données.
- Pour être utilisé dans l'application, un service doit être injecté dans le composant dans lequel on veut l'utiliser.

Services

esiea

- Pour créer votre service, utilisez la commande suivante :

`ng generate service services/cards-service` ou `ng g s services/cards-service`

- Le fichier créé sera chargée de gérer les fiches.

```
@Injectable({
  providedIn: 'root'
})
export class CardsService {
  cards : Card[];

  constructor() {
    this.cards = [
      new Card(1, "Dupont", "Antoine", 1996, "https://static.actu.fr/uploads/2023/10/antoine-dupont-1.png"),
      new Card(2, "Ntamack", "Romain", 1999, "https://static.actu.fr/upload/250x333/romain-ntamack-250.jpg"),
      new Card(3, "Ramos", "Thomas", 1995,  "https://static.actu.fr/uploads/2022/12/OM134361.jpg"),
      new Card(4, "Bielle-Biarrey", "Louis", 2003, "https://static.actu.fr/uploads/2024/bielle-biarrey.jpg"),
    ]
  }

  getAllCards() : Card[] {
    return this.cards;
  }
  // Autres méthodes de gestion des Cards (getbyId, create, update, delete, ...)
}
```

Annotation `@Injectable()` générée automatiquement

Les gestion des cartes est déplacée du `app.ts` vers le service

Services

esiea

- Pour intégrer le service dans un composant, on le déclare comme argument dans son constructeur.
- Intégrons-le dans le composant général de l'application:

```
export class App {
  cards : Card[];
  constructor(private cardService : CardsService) {
    registerLocaleData(frenchLocale);
    this.cards = cardService.getAllCards();
  }
}
```

Le service est injecté avec la visibilité `private` dans le constructeur.

Services : en résumé

esiea

- Le service est l'élément qui centralise les données et offre des méthodes CRUD sur celles-ci à disposition des composants (à comparer à son homonyme en Spring).
- C'est dans celui-ci que l'accès à des données persistantes sera effectué (accès BDD, API, …).
- Les services sont mis à dispositions des composants en les passant en tant que paramètre dans le constructeur du composant.
- C'est l'annotation `@Injectable` générée par Angular qui se charge du reste.